Project Executable Files

Date	01 October 2026
Team ID	SWTID-2026-2430
Project Name	Team Enrollment Confirmation - FitBuddy – AI Fitness Plan Generator using Gemini Models
Maximum Marks	3 Marks

Project Executable Files

This section documents the executable and deployable files for FitBuddy. The project is a FastAPI web application integrated with Google Gemini models, Jinja2 templates, SQLAlchemy and SQLite. The application generates personalized 7-day workout plans, nutrition/recovery tips, and feedback-based plan updates.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / N/A)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / dependency file	Yes
4	Database schema / database files (SQLite, if applicable)	Yes
5	Environment configuration file (.env.example or similar)	Yes
6	Deployed application URL (if hosted)	N/A – Local deployment
7	APK / Executable binary (if applicable for mobile/desktop apps)	N/A – Web application
8	Dockerfile / Containerization config (if applicable)	N/A
9	Test files and test results	To be added after execution
10	Demo video or walkthrough (if required)	To be added

Step 2: File / Folder Structure

The FitBuddy project follows the modular structure documented for the application:

/fitbuddy-project	Project root
/app	Python application files: main.py, routes.py, database.py, Gemini generator modules
/templates	Jinja2 HTML templates: index.html, result.html, all_users.html
/static/images	Static image/assets used by the frontend
requirements.txt	Python dependencies including FastAPI, Uvicorn, Jinja2, SQLAlchemy and Google Generative AI SDK
.env / .env.example	Environment configuration for the Gemini API key
README.md	Setup and run instructions

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	Not provided – application is prepared for local deployment
Login Credentials (Demo)	Not required for the documented local application flow
Platform / Hosting Provider	Local system using FastAPI + Uvicorn
Repository Link	Not provided
Demo Video Link	Not provided

Application Components

Frontend	HTML + Jinja2 templates for user input, generated results, feedback, and admin view.
Backend	FastAPI handles routes, form inputs, validation and application logic.
AI Layer	Google Gemini models generate workout plans, nutrition tips and feedback-based updates.
Database	SQLite with SQLAlchemy stores user details and original/updated plans.

Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to run the project locally:

Pre-requisites: Python and pip installed; access to the Gemini API key; project files available locally.
Step 1: Open a terminal in the FitBuddy project root directory.
Step 2: Create a virtual environment: `python -m venv venv`
Step 3: Activate the environment: `venv\Scripts\activate` (Windows) `source venv/bin/activate` (Linux/macOS)
Step 4: Install dependencies: `pip install -r requirements.txt`
Step 5: Configure the Gemini API key in the environment / .env file: `GOOGLE_API_KEY=your_gemini_api_key_here`
Step 6: Start the FastAPI application: `uvicorn app.main:app --reload`
Step 7: Open the application in a browser at `http://127.0.0.1:8000`. API documentation is available at `/docs`.

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Gemini API access is required for AI-generated plans and tips.	Configure a valid Gemini API key in the environment.
2	Local deployment is documented; no hosted URL was supplied.	Run through Uvicorn on the local machine.
3	Performance measurements are not included until the performance tests are executed.	Run the performance-testing scenarios and record actual values.

Main Application Routes

Route	Purpose
/	Displays the FitBuddy user input form.
/generate-workout	Receives user details and generates the workout plan and nutrition tip.
/submit-feedback	Updates an existing workout plan using user feedback.
/view-all-users	Displays users and their original/updated plans for admin viewing.

Team Work Allocation

Responsibilities are assigned according to the exact order of names provided by the team.

S.No	Team Member	Assigned Work	Main Deliverables
1	Divya.D	Milestone 1 – Model Selection & Architecture	Research Gemini model selection; define system architecture; prepare development environment.
2	Deepika.M	Milestone 2 – Core Functionalities & FastAPI Backend	Develop workout generation, nutrition tip, feedback update, storage functions and backend input processing.
3	Dharani.T	Milestone 3 – routes.py Development	Implement and connect /, /generate-workout, /submit-feedback and /view-all-users routes.
4	Divya shree.R	Milestone 4 – Frontend Development	Develop index.html, result.html and all_users.html; styling, responsiveness and Jinja2 integration.
5	DivyaDharshini.V	Milestone 5 – Deployment & Testing	Prepare local deployment, verify Uvicorn/FastAPI, perform testing, collect evidence and document results.

Project Technology Summary

Technology	Use in FitBuddy
Python / FastAPI	Backend routing, request handling and application logic.
Google Gemini	Workout-plan generation, nutrition/recovery tips and feedback-based plan updates.
Jinja2 + HTML/CSS	Dynamic frontend pages and responsive user interface.
SQLAlchemy + SQLite	Persistent storage of user information and generated plans.
Uvicorn	Local ASGI server used to run the FastAPI application.

Prepared for Team ID SWTID-2026-2430 | Project: Team Enrollment Confirmation - FitBuddy – AI Fitness Plan Generator using Gemini Models